



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
CIENCIA DE LA COMPUTACIÓN
BASES DE DATOS II

DISEÑO E IMPLEMENTACIÓN DE UN MOTOR DE BASE DE DATOS RELACIONAL CON INDEXACIÓN AVANZADA Y CONTROL DE CONCURRENCIA

INTEGRANTES:

Frisancho Gálvez, Gabriel Nicolás (202310080)
Felix Aponte, Renzo Josimar (202220226)
Rojas Leon, Brigitte Dayane (202210498)
Chipayo Flores, Alejandro Sebastian (201910369)
Rondan Cantaro, Lisseth Jomely (202210379)

Proyecto académico presentado como parte del desarrollo del curso
para la implementación de un gestor de bases de datos desde cero.

LIMA, PERÚ
MAYO 2026

Índice

1. Introducción y Objetivo	3
1.1. Propósito del motor de base de datos	3
1.2. Alcance del sistema	3
2. Arquitectura de Almacenamiento y Gestión de Disco OK	3
2.1. Gestión de Páginas	3
2.1.1. Estructura de la Página y Cabecera	3
2.1.2. Operaciones de Bajo Nivel	4
2.2. Definición y Persistencia del Esquema	4
2.2.1. Tipado y Serialización Binaria	4
2.2.2. Mecanismos de Transformación	4
2.3. Monitoreo de Rendimiento de E/S (disk_stats.py)	4
3. Técnicas de Indexación Implementadas	5
3.1. Archivo Secuencial (sequential_file.py)	5
3.1.1. Descripción General	5
3.1.2. Estructura en Disco	5
3.1.3. Algoritmos Implementados	5
3.1.4. Complejidad Teórica	6
3.2. Hashing Extensible (extendible_hashing.py)	6
3.2.1. Descripción General	6
3.2.2. Estructura en Disco	6
3.2.3. Función de Hash	7
3.2.4. Algoritmos Implementados	7
3.2.5. Complejidad Teórica	7
3.3. Árboles B+ (bplus_tree.py)	7
3.3.1. Descripción General	7
3.3.2. Estructura en Disco	8
3.3.3. Algoritmos Implementados	8
3.3.4. Complejidad Teórica	8
3.4. Indexación Espacial con R-Trees (rtree.py)	9
3.4.1. Descripción General	9
3.4.2. Estructura en Disco	9
3.4.3. Función de Distancia (Haversine)	9
3.4.4. Algoritmos Implementados	9
3.4.5. Complejidad Teórica	10
3.5. Resumen de Técnicas de Indexación	10
4. Análisis Teórico Comparativo de Accesos a Disco	10
4.1. Análisis de complejidad I/O para operaciones de búsqueda y rango	10
4.1.1. Sequential File	10
4.1.2. Extendible Hashing	11
4.1.3. B+ Tree	12
4.1.4. R-Tree	12
4.2. Comparativa teórica: Costo de acceso en estructuras arbóreas vs. hash	13
4.2.1. Tabla comparativa de complejidad I/O	13
4.2.2. Análisis comparativo	13
4.2.3. Consideraciones sobre tamaño de página	14

5. Procesamiento de Consultas: Parser SQL	14
5.1. Análisis Léxico	14
5.2. Análisis Sintáctico y AST	16
5.2.1. Construcción del AST	16
5.2.2. Soporte de Gramática SQL	17
5.2.3. Gramática del Parser	18
5.2.4. Validación Sintáctica	20
5.3. Análisis Semántico	20
5.3.1. Validación de Operaciones SQL	20
5.3.2. Verificación de Tipos de Datos	21
5.3.3. Validación de Consultas Espaciales	22
5.3.4. Funciones Auxiliares de Verificación	22
5.3.5. Manejo de Errores Semánticos	23
6. Gestión de Concurrencia y Transacciones	23
6.1. Protocolos de Bloqueo	23
6.2. Planificación de Transacciones	23
6.3. Ciclo de Vida de la Transacción	24
7. Resultados Experimentales y Discusión	24
7.1. Pruebas de rendimiento con volúmenes de datos	24
7.2. Gráficos comparativos y discusión técnica	25
8. Interfaz Gráfica y Capturas	27
8.1. Visualización de la arquitectura y ejecución SQL	27
8.1.1. B+ Tree	27
8.1.2. Extendible Hashing	28
8.1.3. Sequential File	29
8.1.4. R-Tree	29
8.1.5. Manejo de errores	33
9. Conclusiones	33

1. Introducción y Objetivo

1.1. Propósito del motor de base de datos

Este proyecto surge con el objetivo de profundizar en el funcionamiento interno de un sistema gestor de bases de datos (SGBD) sin depender de abstracciones de alto nivel. El propósito central es comprender el costo computacional de la manipulación de datos en memoria secundaria a través de la implementación manual de las estructuras de indexación.

El sistema opera bajo una arquitectura de almacenamiento paginado, donde cada operación de lectura o escritura se ejecuta estrictamente sobre bloques físicos de tamaño fijo (4 KB). Este diseño impone un manejo eficiente de los recursos, priorizando la minimización de accesos a disco (I/O) como métrica principal de rendimiento, ya que cada interacción con el almacenamiento implica la gestión de páginas completas en lugar de registros individuales.

1.2. Alcance del sistema

El motor integra diversos componentes modulares diseñados para resolver problemas específicos de recuperación de información:

- **Indexación Especializada:** Se implementan estructuras optimizadas para distintos casos de uso: el **B+ Tree** para la gestión de rangos y claves primarias ordenadas, el **Hash Extensible** para búsquedas exactas con costo constante, y el **R-Tree** para el procesamiento de datos multidimensionales mediante consultas espaciales (radio y KNN).
- **Procesamiento de Consultas (Parser):** El motor incluye una interfaz SQL capaz de traducir sentencias en un Árbol de Sintaxis Abstracta (AST), permitiendo una ejecución directa sobre las estructuras de indexación del sistema.
- **Control de Concurrency:** Se incorpora un simulador para la gestión de transacciones simultáneas, permitiendo el registro y la identificación de conflictos en el acceso a datos compartidos.

2. Arquitectura de Almacenamiento y Gestión de Disco OK

La capa de almacenamiento es el núcleo del motor de base de datos, encargándose de la persistencia y la recuperación de información mediante una abstracción de bajo nivel sobre el sistema de archivos.

2.1. Gestión de Páginas

La clase **PageManager** implementa un modelo de almacenamiento basado en páginas de tamaño fijo de 4096 bytes (4 KB). Esta decisión de diseño es crítica para optimizar la transferencia de datos entre la memoria principal y el disco, alineándose con el tamaño de bloque estándar en la mayoría de los sistemas operativos modernos.

2.1.1. Estructura de la Página y Cabecera

Cada página se gestiona como una unidad independiente que incluye una cabecera (*header*) de 2 bytes.

- **Formato de Cabecera:** Se utiliza un formato **big-endian unsigned short (>H)** para almacenar el conteo de registros por página, asegurando la interoperabilidad del archivo binario.

- **Cálculo de Offsets:** El acceso a una página específica se realiza mediante un cálculo de desplazamiento lineal: $\text{Offset} = \text{page_id} \times \text{PAGE_SIZE}$.

2.1.2. Operaciones de Bajo Nivel

El sistema implementa tres operaciones fundamentales que son utilizadas por todos los índices del motor:

- **Lectura (`read_page`):** Extrae bloques exactos de datos. Si el bloque leído es menor al tamaño de página, se aplica un relleno (*padding*) con ceros binarios (`b" \x00"`).
- **Escritura (`write_page`):** Persiste los datos asegurando que no superen el límite de 4 KB. Cada operación incrementa el contador de escrituras en la clase `DiskStats`.
- **Asignación (`allocate_page`):** Permite el crecimiento dinámico del archivo añadiendo páginas vacías al final del mismo, lo que facilita la expansión de índices como el B+ Tree y el Hashing Extensible.

El sistema prioriza el análisis del costo de entrada y salida (*I/O*). Mediante el objeto `stats`, se registra cada lectura y escritura física en el `PageManager`, permitiendo validar el rendimiento teórico (Big O) frente a la carga real en disco.

2.2. Definición y Persistencia del Esquema

El archivo `schema.py` define la estructura lógica de las tablas y garantiza que cada registro ocupe un tamaño fijo (*fixed-length records*) en disco, facilitando el cálculo de *offsets*.

2.2.1. Tipado y Serialización Binaria

El motor soporta tipos de datos elementales con tamaños predefinidos para asegurar la portabilidad:

- **Tipos Estáticos:** INT (4B), FLOAT (8B) y BOOL (1B), gestionados mediante el módulo `struct` con formato *big-endian* (`>i`, `>d`, `>?`).
- **Tipo Variable:** VARCHAR requiere una longitud máxima definida por el usuario, la cual determina su tamaño fijo en disco mediante técnicas de *padding*.

2.2.2. Mecanismos de Transformación

- **Serialización (`serialize`):** Convierte diccionarios de Python en una cadena de bytes continua de tamaño `record_size`. Los campos ausentes o cadenas cortas se rellenan con ceros binarios.
- **Deserialización (`deserialize`):** Realiza el proceso inverso, decodificando los bytes leídos de las páginas y eliminando el *padding* en campos de texto (`rstrip`) para recuperar la información original.

2.3. Monitoreo de Rendimiento de E/S (`disk_stats.py`)

El módulo `disk_stats.py` implementa una unidad de monitoreo para cuantificar el costo operacional de cada consulta.

- **Métricas de Transferencia:** Registra de forma acumulativa las operaciones de lectura (`reads`) y escritura (`writes`) ejecutadas por el `PageManager`.

- **Métricas Temporales:** Calcula el tiempo de ejecución en milisegundos (`time_ms`) con alta precisión, permitiendo correlacionar el tiempo de CPU con el número de accesos a páginas.
- **Validación Experimental:** Mediante el método `snapshot()`, el sistema captura el estado del motor antes y después de cada sentencia SQL, proporcionando la data necesaria para la sección de resultados experimentales y la comparativa de eficiencia teórica.

3. Técnicas de Indexación Implementadas

3.1. Archivo Secuencial (`sequential_file.py`)

3.1.1. Descripción General

El Secuencial File mantiene los registros ordenados por clave primaria en un archivo principal. Para optimizar las inserciones y evitar reorganizaciones costosas en cada operación, se utiliza un **archivo auxiliar de desbordamiento** donde se almacenan temporalmente los nuevos registros. Cuando el archivo auxiliar acumula un umbral de $K = 10$ registros, se ejecuta una **reconstrucción física** que fusiona ambos archivos, generando un nuevo archivo principal ordenado y vaciando el auxiliar.

3.1.2. Estructura en Disco

El Sequential File utiliza dos archivos de páginas:

- **Archivo Principal (.bin):** Mantiene los registros ordenados por clave primaria.
- **Archivo Auxiliar (.aux.bin):** Almacena inserciones recientes sin ningún orden.

Formato de página (ambos archivos):

[2B contador] [flag(1B)] [registro1_bytes] [flag(1B)] [registro2_bytes] ...

- **Cabecera (2 bytes):** Número total de registros en la página (activos + eliminados)
- **Flag (1 byte):** 0x00 = activo, 0xFF = eliminado
- **Umbral K:** 10 registros en el auxiliar antes de disparar reconstrucción

3.1.3. Algoritmos Implementados

Búsqueda Binaria (`_binary_search`): Aplica búsqueda binaria sobre las páginas del archivo principal usando la primera clave activa de cada página. Retorna el identificador de la página donde debería estar la clave buscada.

Inserción (`add`): Verifica que la clave no exista. Busca la página destino en el principal. Si tiene espacio, inserta manteniendo el orden. Si no, añade al archivo auxiliar. Cuando el auxiliar alcanza 10 registros, ejecuta una reconstrucción que fusiona ambos archivos.

Búsqueda Puntual (`search`): Localiza la página candidata con búsqueda binaria y revisa páginas adyacentes. Busca la clave activa en el principal. Si no la encuentra, recorre secuencialmente el archivo auxiliar. Retorna el registro o None.

Búsqueda por Rango (range_search): Encuentra la página inicial con búsqueda binaria. Recorre las páginas del principal agregando registros que estén en el rango [begin, end]. Luego recorre todo el auxiliar aplicando el mismo filtro. Ordena los resultados y los retorna.

Eliminación (remove): Busca la clave en el principal. Si la encuentra activa, la marca como eliminada (0xFF). Si no está en el principal, busca en el auxiliar y la marca como eliminada. Retorna True si se eliminó, False en caso contrario.

Reconstrucción (_rebuild): Recopila todos los registros activos del principal y del auxiliar. Los fusiona y ordena por clave. Elimina el archivo principal antiguo y escribe los registros ordenados en nuevas páginas. Vacía el archivo auxiliar y reinicia el contador.

3.1.4. Complejidad Teórica

Operación	Sequential File
Búsqueda puntual	$O(\log_2 P + R)$
Búsqueda por rango	$O(\log_2 P + S + R)$
Inserción	$O(1)$ en auxiliar, $O(P)$ durante rebuild
Eliminación	$O(\log_2 P + R)$

Donde: P = número de páginas, R = registros en auxiliar, S = registros en el rango

Cuadro 1: Complejidad del Archivo Secuencial

3.2. Hashing Extensible (extendible_hashing.py)

3.2.1. Descripción General

El Extendible Hashing es una estructura de hash dinámico que utiliza un directorio para mapear claves a buckets (páginas). El directorio crece dinámicamente mediante un proceso llamado *split de buckets*. Cada bucket tiene una profundidad local, mientras que el directorio tiene una profundidad global. Esta técnica garantiza búsquedas en tiempo constante para claves exactas, pero no soporta búsquedas por rango.

3.2.2. Estructura en Disco

El Hashing Extensible utiliza dos archivos principales:

- **Directorio (archivo .dir.bin):** Almacena la profundidad global y el mapeo de hashes a páginas bucket.
- **Buckets (archivo .bin):** Almacenan los registros físicos junto con flags de estado.

Formato del Directorio:

[global_depth(4B)] [page_id_0(4B)] [page_id_1(4B)] [page_id_2(4B)] ...

Formato del Bucket (página):

[2B contador] [flag(1B)] [registro1_bytes] [flag(1B)] [registro2_bytes] ...

- **global_depth:** Profundidad global del directorio (número de bits)
- **page_id:** Identificador de la página bucket (4 bytes)
- **flag:** 0x01 = activo, 0x00 = eliminado
- **BUCKET_SIZE:** 4 registros máximos por bucket antes de split

3.2.3. Función de Hash

La función hash implementada garantiza un valor positivo de 31 bits y extrae los bits menos significativos según la profundidad solicitada:

```
_hash(key, depth):  
    hash_val = hash(key) & 0x7FFFFFFF # entero positivo  
    mask = (1 << depth) - 1  
    return hash_val & mask
```

3.2.4. Algoritmos Implementados

Inserción (add): Verifica que la clave no exista. Lee el directorio, calcula el hash de la clave y localiza el bucket destino. Busca un slot eliminado para reutilizarlo o agrega el registro al final. Si el bucket supera los 4 registros, ejecuta un split que divide el bucket y redistribuye los registros. En caso contrario, escribe el bucket actualizado.

Búsqueda (search): Lee el directorio, calcula el hash de la clave y obtiene el bucket correspondiente. Recorre secuencialmente los registros del bucket y retorna aquel cuya coincida la clave y esté activo. Si no lo encuentra, retorna None.

Eliminación (remove): Localiza el bucket mediante el hash de la clave. Recorre sus registros y si encuentra la clave activa, cambia su flag a eliminado (0x00) y escribe el bucket. Retorna True si se eliminó, False en caso contrario.

Split de Bucket (_split_bucket): Identifica todos los índices del directorio que apuntan al bucket desbordado. Calcula la profundidad local y la incrementa en uno. Crea un nuevo bucket y redistribuye los registros según el nuevo bit significativo. Si es necesario, duplica el directorio. Actualiza las entradas del directorio para que apunten a los buckets correctos.

3.2.5. Complejidad Teórica

Operación	Extendible Hashing
Búsqueda	$O(1)$ promedio, $O(\text{BUCKET_SIZE})$ peor caso
Búsqueda por rango	No soportado (lanza <code>NotSupportedError</code>)
Inserción	$O(1)$ promedio, split ocasional $O(\text{BUCKET_SIZE})$
Eliminación	$O(1)$ promedio

Cuadro 2: Complejidad del Hashing Extensible

3.3. Árboles B+ (bplus_tree.py)

3.3.1. Descripción General

El Árbol B+ es una estructura de indexación balanceada que mantiene todos los registros en las hojas, mientras que los nodos internos almacenan únicamente claves para guiar la búsqueda. Las hojas se encuentran enlazadas entre sí mediante punteros `next_leaf`, lo que permite recorridos secuenciales eficientes para búsquedas por rango. Cada nodo tiene una capacidad máxima de 3 claves (grado 4), y cuando se desborda se divide en dos nodos, promoviendo la clave media hacia el nodo padre.

3.3.2. Estructura en Disco

Cada nodo del árbol ocupa una página de 4 KB con el siguiente formato:

[is_leaf(1B)] [num_keys(2B)] [next_leaf(4B)] [keys...] [children/records...]

- **is_leaf:** 1 = hoja, 0 = nodo interno
- **num_keys:** cantidad de claves en el nodo
- **next_leaf:** puntero a la siguiente hoja (solo en hojas)
- **keys:** claves ordenadas de tamaño fijo según el tipo
- **children:** identificadores de página (nodos internos) o registros serializados (hojas)

3.3.3. Algoritmos Implementados

Búsqueda Puntual (search): Localiza la hoja que podría contener la clave mediante descenso desde la raíz. En cada nodo interno, usa búsqueda binaria para elegir el hijo correcto. Una vez en la hoja, recorre secuencialmente las claves y retorna el registro si la clave coincide.

Búsqueda por Rango (range_search): Encuentra la hoja inicial correspondiente a la clave **begin** mediante descenso normal. Luego recorre las hojas enlazadas por **next_leaf**, recolectando todos los registros cuyas claves estén en el rango [**begin**, **end**]. La exploración se detiene cuando la clave supera **end**.

Inserción (add): Desciende hasta la hoja correspondiente e inserta el registro manteniendo el orden. Si la hoja supera la capacidad máxima (3 claves), se divide en dos hojas, se promueve la clave media al nodo padre y se actualiza el enlace **next_leaf**. Si la división se propaga hasta la raíz, se crea un nuevo nodo raíz.

Eliminación (remove): Desciende hasta la hoja que contiene la clave y la elimina. Si la hoja queda por debajo del mínimo de registros, intenta redistribuir con un hermano o fusiona ambos nodos. En caso de fusión, se elimina la clave de separación del nodo padre. Si el nodo padre también queda vacío, la fusión se propaga hacia arriba.

Eliminación de Referencia (remove_ref): Busca mediante **iter_record_refs()** el registro que contiene la clave foránea especificada y elimina el registro correspondiente del índice.

3.3.4. Complejidad Teórica

Operación	B+ Tree
Búsqueda puntual	$O(\log_F N)$
Búsqueda por rango	$O(\log_F N + S)$
Inserción	$O(\log_F N)$
Eliminación	$O(\log_F N)$

Donde: F = factor de ramificación, N = número de registros, S = registros en rango

Cuadro 3: Complejidad del B+ Tree

3.4. Indexación Espacial con R-Trees (rtree.py)

3.4.1. Descripción General

El R-Tree es una estructura de indexación diseñada para datos espaciales, organizando puntos geográficos (latitud, longitud) mediante Rectángulos Mínimos de Contención (MBR). Cada nodo almacena un MBR que engloba a todos sus descendientes. El árbol mantiene un balance aproximado mediante divisiones cuando un nodo se desborda, utilizando el criterio de *squared minimal area increase* para elegir la rama de inserción. Soporta búsquedas por radio (range search) y por k vecinos más cercanos (kNN).

3.4.2. Estructura en Disco

Cada nodo del R-Tree ocupa una página de 4 KB. Existen dos tipos de nodos:

Nodos Hoja:

```
[is_leaf(1)][count(2)][lat(8)][lon(8)][pk_bytes]...
```

Nodos Internos:

```
[is_leaf(0)][count(2)][min_lat(8)][min_lon(8)][max_lat(8)][max_lon(8)][child(4)]...
```

- **is_leaf:** 1 para hojas, 0 para nodos internos
- **count:** número de entradas en el nodo
- **MBR:** (min_lat, min_lon, max_lat, max_lon) en nodos internos
- **child:** identificador de página hija
- **pk:** clave primaria serializada en hojas

3.4.3. Función de Distancia (Haversine)

La distancia entre dos puntos geográficos se calcula mediante la fórmula del haversine:

$$\text{dist} = 6371 \cdot 2 \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \text{lat}}{2} \right) + \cos(\text{lat}_1) \cdot \cos(\text{lat}_2) \cdot \sin^2 \left(\frac{\Delta \text{lon}}{2} \right)} \right)$$

El resultado se expresa en kilómetros.

3.4.4. Algoritmos Implementados

Búsqueda por Radio (range_search): Recorre el árbol desde la raíz. Si un nodo interno tiene un MBR que intersecta el círculo definido por el punto y el radio, explora sus hijos. En las hojas, verifica la distancia haversine de cada punto y lo agrega a los resultados si está dentro del radio.

K Vecinos Más Cercanos (knn): Utiliza una cola de prioridad ordenada por distancia. Comienza con la raíz y expande el nodo más cercano primero. Para nodos internos, calcula la distancia desde el punto de consulta al MBR (usando el punto más cercano del rectángulo). Para hojas, calcula la distancia real a cada punto. Extrae los k puntos con menor distancia.

Inserción (add_ref): Desciende desde la raíz eligiendo en cada nodo interno el hijo cuya expansión del MBR sea mínima. Al llegar a una hoja, inserta el punto. Si la hoja supera **max_entries** (50 registros), se divide mediante el algoritmo de *pick split seeds* que busca maximizar el área desperdiciada. Si la división se propaga, se ajustan los MBRs hacia arriba.

Eliminación (remove_ref): Busca el punto que contiene la clave primaria especificada. Lo elimina de la hoja. Si la hoja queda por debajo del mínimo de registros (25), sus puntos se reinsertan en el árbol como nuevos puntos.

3.4.5. Complejidad Teórica

Operación	R-Tree
Búsqueda por radio	$O(\log_M N + R)$
KNN	$O(\log_M N + K \cdot \log K)$
Inserción	$O(\log_M N)$
Eliminación	$O(\log_M N)$

Donde: M = factor de ramificación, N = número de puntos, R = puntos dentro del radio

Cuadro 4: Complejidad del R-Tree

3.5. Resumen de Técnicas de Indexación

Cuadro 5: Comparativa de Técnicas de Indexación en el Motor de Base de Datos

Técnica	Caso de Uso Ideal en el Motor	Ventaja Principal	Complejidad
Seq. File	<i>Full Table Scans</i> y carga masiva de datos iniciales.	Simplicidad y Orden	$O(\log n)$
Extendible Hash	Búsquedas por igualdad exacta y Joins de alta velocidad.	Acceso constante	$O(1)$
B+ Tree	Consultas por rango (<i>BETWEEN</i>) y Claves Primarias agrupadas.	Versatilidad y Orden	$O(\log_f n)$
R-Tree	Atributos geospaciales y consultas de proximidad (GIS).	Filtrado espacial	Variable

4. Análisis Teórico Comparativo de Accesos a Disco

4.1. Análisis de complejidad I/O para operaciones de búsqueda y rango

El costo de acceso a disco es la métrica fundamental en un sistema gestor de bases de datos, ya que la transferencia de páginas entre memoria secundaria y principal representa el cuello de botella principal. A continuación se analiza la complejidad teórica en términos de accesos a página para cada estructura implementada.

4.1.1. Sequential File

El Sequential File mantiene los registros ordenados en un archivo principal y utiliza un archivo auxiliar para nuevas inserciones. Cada página tiene capacidad para $R_p = \lfloor (4096 - 2) / (\text{record_size} + 1) \rfloor$ registros, donde 2 bytes corresponden al contador y 1 byte al flag de estado.

Búsqueda:

- Archivo principal: búsqueda binaria sobre páginas $\lceil \log_2 P \rceil$ accesos, donde P es el número de páginas

- Archivo auxiliar: escaneo lineal de hasta A páginas, donde A depende del número de registros en el auxiliar
- Costo total: $O(\log_2 P + A)$ accesos a página

Búsqueda por rango:

- Localización de la página inicial: $\lceil \log_2 P \rceil$ accesos
- Recorrido secuencial de páginas: S_p páginas que contienen registros en el rango
- Escaneo completo del archivo auxiliar: A páginas
- Costo total: $O(\log_2 P + S_p + A)$ accesos

Inserción:

- Inserción directa en el principal: $O(1)$ si hay espacio
- Inserción en auxiliar: $O(1)$ (append al final)
- Reconstrucción (cuando auxiliar alcanza $K = 10$ registros): requiere leer todas las páginas del principal y del auxiliar, y escribirlas nuevamente ordenadas. Costo $O(P + A)$ accesos, pero ocurre cada K inserciones.

4.1.2. Extendible Hashing

El Extendible Hashing utiliza un directorio dinámico que mapea códigos hash a buckets. Cada bucket tiene capacidad fija de $B = 4$ registros. El directorio tiene profundidad global g , por lo que contiene 2^g entradas de 4 bytes cada una, ocupando una única página hasta que $2^g \times 4 > 4096$.

Búsqueda:

- Lectura del directorio: 1 acceso (el directorio cabe en una página hasta $g = 10$)
- Lectura del bucket: 1 acceso
- Escaneo dentro del bucket: hasta $B = 4$ comparaciones en memoria
- Costo total: $O(1)$ accesos a página (constante)

Búsqueda por rango: No soportada por diseño. El hashing no mantiene ningún orden entre claves.

Inserción:

- Localización del bucket: 2 accesos (directorio + bucket)
- Si el bucket tiene espacio: 1 escritura adicional
- Si el bucket se desborda (> 4 registros): se ejecuta un split que implica:
 - Crear un nuevo bucket (1 escritura)
 - Redistribuir $B + 1$ registros entre dos buckets
 - Posible duplicación del directorio (escritura de la página de directorio)
- Costo amortizado: $O(1)$ accesos

4.1.3. B+ Tree

El B+ Tree organiza los datos en nodos de tamaño fijo. Cada nodo tiene capacidad máxima de 3 claves (grado 4), pues $3 \times \text{key_size} + 4 \times 4 + \text{overhead} \leq 4096$. Las hojas están enlazadas mediante punteros `next_leaf`.

Búsqueda:

- Se recorren h niveles desde la raíz hasta la hoja
- Altura $h = \lceil \log_4(N) \rceil$, donde N es el número de registros
- Cada nivel requiere 1 acceso a página para leer el nodo
- Costo total: $O(\log_4 N)$ accesos a página

Búsqueda por rango:

- Localización de la hoja inicial: $O(\log_4 N)$ accesos
- Recorrido de L hojas mediante punteros `next_leaf`: L accesos
- Costo total: $O(\log_4 N + L)$ accesos, donde L es el número de hojas que contienen registros en el rango

Inserción:

- Descenso para localizar la hoja: $O(\log_4 N)$ accesos de lectura
- Inserción en la hoja: 1 escritura
- Si ocurre split: propagación hacia arriba, afectando hasta $O(\log_4 N)$ nodos
- Costo amortizado: $O(\log_4 N)$ accesos

4.1.4. R-Tree

El R-Tree organiza puntos espaciales mediante Rectángulos Mínimos de Contención (MBR). Cada nodo tiene capacidad máxima de 50 entradas y mínima de 25 entradas (excepto la raíz).

Búsqueda por radio (range_search):

- Se recorre el árbol desde la raíz, explorando solo los nodos cuyo MBR intersecta el círculo de búsqueda
- En el peor caso (consulta que cubre toda el área): $O(P)$ accesos, donde P es el número de páginas
- En el caso promedio, la poda por MBR reduce significativamente los accesos: $O(\log_{50} N + R_p)$, donde R_p es el número de páginas hoja en el área de búsqueda

KNN (k vecinos más cercanos):

- Utiliza una cola de prioridad para expandir los nodos más cercanos primero
- Complejidad similar a la búsqueda por radio, con sobrecarga adicional por la cola de prioridad
- Costo teórico: $O(\log_{50} N + K \cdot \log K)$ accesos en el mejor caso

Inserción:

- Descenso eligiendo el hijo con menor expansión del MBR: $O(\log_{50} N)$ accesos de lectura
- Inserción en la hoja: 1 escritura
- Si ocurre split (> 50 entradas): se dividen las entradas en dos nodos, propagando el ajuste de MBRs hacia arriba
- Costo amortizado: $O(\log_{50} N)$ accesos

4.2. Comparativa teórica: Costo de acceso en estructuras arbóreas vs. hash

4.2.1. Tabla comparativa de complejidad I/O

Operación	Sequential File	Extendible Hashing	B+ Tree	R-Tree
Búsqueda	$O(\log_2 P + A)$	$O(1)$	$O(\log_4 N)$	$O(\log_{50} N)$
Búsqueda por rango	$O(\log_2 P + S_p + A)$	No soportado	$O(\log_4 N + L)$	$O(\log_{50} N + R_p)$
Inserción	$O(1)$ amortizado	$O(1)$ amortizado	$O(\log_4 N)$	$O(\log_{50} N)$
Eliminación	$O(\log_2 P + A)$	$O(1)$	$O(\log_4 N)$	$O(\log_{50} N)$

Cuadro 6: Comparativa de complejidad de accesos a página

4.2.2. Análisis comparativo

Ventajas y desventajas de cada estructura:

- **Extendible Hashing:** Óptimo para búsquedas puntuales con $O(1)$ accesos, pero no soporta búsquedas por rango. Ideal para sistemas donde predominan consultas exactas por clave, como tablas de catálogos o diccionarios.
- **B+ Tree:** Ofrece un balance excelente entre búsquedas puntuales y por rango. Su altura logarítmica garantiza pocos accesos incluso para grandes volúmenes de datos. El enlace entre hojas permite recorridos secuenciales eficientes.
- **Sequential File:** La búsqueda binaria por páginas es eficiente, pero la presencia del archivo auxiliar degrada las búsquedas cuando hay muchas inserciones pendientes. La reconstrucción periódica es costosa pero mantiene el orden.
- **R-Tree:** Especializado para datos espaciales. La poda por MBR es efectiva para filtrar nodos irrelevantes, pero el rendimiento depende de la calidad de los MBRs y la distribución de los datos.

Caso de uso por tipo de consulta:

- **Consultas exactas:** Extendible Hashing es superior con $O(1)$ accesos.
- **Consultas por rango escalar:** B+ Tree y Sequential File son adecuados, pero B+ Tree mantiene $O(\log N)$ incluso con inserciones frecuentes.
- **Consultas espaciales:** R-Tree es la única estructura que las soporta eficientemente mediante poda por MBR.
- **Carga masiva de datos:** Sequential File con reconstrucción periódica puede ser eficiente si se ajusta el umbral K adecuadamente.

4.2.3. Consideraciones sobre tamaño de página

El tamaño de página fijo de 4096 bytes impacta directamente en la capacidad de cada estructura:

- **Sequential File:** $R_p = \lfloor 4094 / (record_size + 1) \rfloor$ registros por página
- **Extendible Hashing:** $B = 4$ registros por bucket (independiente de *record_size*)
- **B+ Tree:** Nodos de 3 claves (limitado por *PAGE_SIZE*)
- **R-Tree:** 50 entradas por nodo (limitado por *PAGE_SIZE*)

La elección del tamaño de página es un compromiso entre el número de accesos (páginas pequeñas = más accesos) y el desperdicio de espacio (páginas grandes = más registros no utilizados por página). El valor de 4 KB es estándar en sistemas de archivos modernos y se alinea con el tamaño de bloque típico.

5. Procesamiento de Consultas: Parser SQL

El procesamiento de consultas constituye uno de los componentes centrales del sistema gestor de bases de datos, ya que permite interpretar y validar las sentencias SQL ingresadas por el usuario antes de su ejecución. En esta implementación, el parser fue diseñado siguiendo una arquitectura modular basada en tres etapas fundamentales: análisis léxico, análisis sintáctico y análisis semántico. Cada una de estas fases cumple una responsabilidad específica dentro del flujo de procesamiento de consultas, permitiendo transformar una cadena de texto SQL en una representación estructurada y validada lista para ser ejecutada por el motor de base de datos.

La arquitectura del parser fue implementada mediante una clase fachada denominada **Parser**, encargada de coordinar internamente las tres fases del análisis. Durante la ejecución, el sistema recibe una consulta SQL como entrada, realiza la tokenización de la sentencia mediante el analizador léxico, construye posteriormente una representación sintáctica utilizando el analizador sintáctico y, finalmente, ejecuta un proceso de validación semántica sobre el árbol resultante antes de autorizar su ejecución. Esta separación por etapas permite mantener un diseño desacoplado, extensible y más sencillo de mantener.

Adicionalmente, el parser incorpora un mecanismo unificado de manejo de errores mediante excepciones personalizadas, permitiendo detectar y reportar de manera precisa errores léxicos, sintácticos y semánticos durante el procesamiento de consultas.

5.1. Análisis Léxico

El análisis léxico representa la primera etapa del procesamiento de consultas SQL dentro del sistema. Su objetivo principal consiste en transformar la sentencia SQL ingresada por el usuario en una secuencia estructurada de unidades léxicas denominadas tokens, las cuales serán posteriormente utilizadas por el analizador sintáctico para construir la representación interna de la consulta.

La implementación desarrollada utiliza un analizador léxico manual basado en un recorrido secuencial de la cadena de entrada. Este enfoque permite un control preciso sobre la gramática soportada por el motor y facilita la integración de nuevas instrucciones SQL específicas del proyecto. El componente fue implementado mediante la clase **LexicalAnalyzer**, encargada de recorrer carácter por carácter la consulta utilizando un puntero de posición (`self.pos`) y un contador de líneas (`self.line`) para el manejo de errores y depuración.

El proceso de tokenización sigue una estrategia *single-pass*, recorriendo la consulta únicamente una vez con una complejidad temporal aproximada de $\mathcal{O}(n)$, donde n representa la

longitud de la cadena de entrada. Durante este recorrido, el analizador identifica distintos tipos de elementos léxicos como palabras reservadas, identificadores, literales, operadores y delimitadores.

La clasificación de tokens se implementó mediante el enumerado `TokenType`, el cual centraliza todos los símbolos reconocidos por el subconjunto SQL soportado por el sistema. Los tokens definidos en el proyecto se agrupan en las siguientes categorías:

■ **Palabras clave SQL:**

- CREATE, TABLE, DROP
- SELECT, FROM, WHERE
- INSERT, INTO, VALUES
- DELETE, INDEX
- PRIMARY, KEY
- ON, USING

■ **Palabras clave de consultas de búsqueda:**

- POINT, RADIUS, K
- BETWEEN, IN, AND

■ **Tipos de índices soportados:**

- SEQUENTIAL
- EXTENDIBLE HASHING
- BPLUS TREE
- RTREE

■ **Tipos de datos:**

- Numéricos: INT, INTEGER, SMALLINT, BIGINT, REAL, DOUBLE PRECISION
- Lógicos: BOOLEAN
- Texto: CHAR[n]
- Temporales: DATE, TIME

■ **Literales e identificadores:**

- Identificadores (IDENTIFIER)
- Literales enteros (INTEGER_LITERAL)
- Literales flotantes (FLOAT_LITERAL)
- Literales de cadena (STRING_LITERAL)
- Valores booleanos (TRUE, FALSE)

■ **Operadores relacionales:**

- =, <, >
- <=, >=

■ **Símbolos y delimitadores:**

- Paréntesis: ()

- Separadores: ,
- Terminador de sentencia: ;
- Operador universal: *

El analizador incorpora además mecanismos especializados para el reconocimiento de distintos tipos de literales. Las cadenas de texto son procesadas mediante el método `read_string()`, permitiendo el manejo de caracteres escapados utilizando comillas dobles consecutivas. Asimismo, el método `read_number()` soporta tanto números enteros como valores decimales y negativos, diferenciando automáticamente entre tokens enteros y flotantes.

Para resolver ambigüedades léxicas, el sistema implementa una técnica de *lookahead* mediante la función `peek()`, utilizada principalmente en el reconocimiento de operadores compuestos como `<=` y `>=`. Este mecanismo permite inspeccionar caracteres futuros sin consumirlos inmediatamente, garantizando una tokenización correcta.

Finalmente, el analizador léxico incorpora un mecanismo robusto de manejo de errores mediante la excepción personalizada `LexError`. Cuando se detecta un carácter inválido o una cadena sin cierre, el sistema detiene inmediatamente el análisis e informa la línea exacta donde ocurrió el problema, facilitando el proceso de depuración y validación de consultas SQL.

5.2. Análisis Sintáctico y AST

El análisis sintáctico constituye la segunda fase del procesamiento de consultas SQL y tiene como objetivo verificar que la secuencia de tokens generada por el analizador léxico cumpla correctamente con la gramática definida por el sistema. Durante esta etapa, el parser transforma la secuencia lineal de tokens en una representación estructurada denominada **AST** (*Abstract Syntax Tree*), la cual modela internamente la consulta SQL mediante nodos especializados.

La implementación fue desarrollada mediante la clase `SyntacticAnalyzer`, siguiendo una estrategia de análisis descendente recursivo. Este enfoque permite implementar manualmente cada regla gramatical mediante funciones específicas, proporcionando un mayor control sobre la sintaxis soportada y facilitando la extensión del lenguaje SQL utilizado por el proyecto. El parser mantiene internamente una lista de tokens y un puntero de posición (`self.pos`) para recorrer secuencialmente la entrada tokenizada.

El proceso principal de análisis se realiza mediante el método `parse()`, el cual identifica dinámicamente el tipo de sentencia SQL a partir del primer token de entrada y redirige el procesamiento hacia la rutina sintáctica correspondiente. Actualmente, el sistema soporta instrucciones `CREATE`, `DROP`, `INSERT`, `SELECT` y `DELETE`.

5.2.1. Construcción del AST

El resultado del análisis sintáctico es un árbol sintáctico abstracto compuesto por nodos especializados que representan cada tipo de operación SQL soportada. Estos nodos encapsulan únicamente la información estructural relevante para la ejecución de la consulta, eliminando detalles léxicos innecesarios como delimitadores o símbolos auxiliares.

La arquitectura AST implementada en el proyecto incluye los siguientes tipos de nodos:

■ Nodos de definición de estructuras:

- `CreateTableNode`
- `CreateIndexNode`
- `DropTableNode`
- `DropIndexNode`

■ Nodos de manipulación de datos:

- InsertNode
- DeleteNode
- **Nodos de consulta SELECT:**
 - SelectAllNode
 - SelectEqualNode
 - SelectComparisonNode
 - SelectRangeNode
 - SelectPointRadiusNode
 - SelectKNNNode
- **Nodos de literales tipados:**
 - DateLiteralNode
 - TimeLiteralNode

Cada nodo almacena únicamente la información necesaria para representar semánticamente la operación SQL. Por ejemplo, el nodo `CreateTableNode` contiene el nombre de la tabla, la definición de columnas y opcionalmente la ruta de importación desde archivo, mientras que `CreateIndexNode` almacena el nombre del índice, la tabla asociada, las columnas indexadas y la técnica de indexación seleccionada.

5.2.2. Soporte de Gramática SQL

El parser implementa un subconjunto especializado de SQL orientado al motor de almacenamiento desarrollado en el proyecto. Entre las construcciones sintácticas soportadas destacan:

- **Creación de tablas con llave primaria:**

```
CREATE TABLE tabla (
columna TIPO PRIMARY KEY USING BPLUS TREE
);
```

- **Creación de índices secundarios:**

```
CREATE INDEX idx_nombre
ON tabla(columna)
USING RTREE;
```

- **Inserción de registros:**

```
INSERT INTO tabla VALUES (...);
```

- **Consultas SELECT simples y condicionales:**

```
SELECT * FROM tabla;
SELECT * FROM tabla WHERE edad >= 18;
SELECT * FROM tabla WHERE fecha BETWEEN ...;
```

- **Consultas espaciales:**

```
SELECT * FROM tabla
WHERE ubicacion IN (POINT(x,y), RADIUS r);
```

```
SELECT * FROM tabla
WHERE ubicacion IN (POINT(x,y), K k);
```

El soporte de consultas espaciales representa una extensión relevante del parser, permitiendo integrar búsquedas geoespaciales y consultas KNN (*k-nearest neighbors*) directamente dentro de la gramática SQL del sistema. Estas instrucciones son traducidas internamente hacia nodos AST especializados como `SelectPointRadiusNode` y `SelectKNNNode`

5.2.3. Gramática del Parser

La gramática implementada por el parser define formalmente el subconjunto de instrucciones SQL que puede reconocer el sistema. Esta gramática está orientada a operaciones de definición de datos, manipulación de datos, consultas condicionales, consultas por rango, consultas espaciales e instrucciones de administración de índices.

A continuación, se presenta la gramática en una notación similar a EBNF:

```
<statement> ::= <create_statement> ";"
               | <drop_statement> ";"
               | <insert_statement> ";"
               | <select_statement> ";"
               | <delete_statement> ";"

<create_statement> ::= <create_table>
                      | <create_index>

<create_table> ::= "CREATE" "TABLE" <identifier>
                  "(" <column_definition_list> ")"
                  [ "FROM" "FILE" <string_literal> ]

<column_definition_list> ::= <column_definition>
                             | <column_definition> "," <column_definition_list>

<column_definition> ::= <identifier> <data_type>
                       [ "PRIMARY" "KEY" [ "USING" <primary_index_type> ] ]

<create_index> ::= "CREATE" "INDEX" <identifier>
                  "ON" <identifier>
                  "(" <identifier> [ "," <identifier> ] ")"
                  "USING" <index_type>

<drop_statement> ::= "DROP" "TABLE" <identifier>
                    | "DROP" "INDEX" <identifier> "ON" <identifier>

<insert_statement> ::= "INSERT" "INTO" <identifier>
                      "VALUES" "(" <literal_list> ")"

<literal_list> ::= <literal>
                  | <literal> "," <literal_list>
```

```

<select_statement> ::= "SELECT" "*" "FROM" <identifier>
                    | "SELECT" "*" "FROM" <identifier>
                      "WHERE" <where_condition>

<where_condition> ::= <identifier> "=" <literal>
                    | <identifier> <comparison_operator> <literal>
                    | <identifier> "BETWEEN" <literal> "AND" <literal>
                    | <identifier> "IN" "(" "POINT" "(" <literal> "," <literal> ")"
                      "," "RADIUS" <literal> ")"
                    | <identifier> "IN" "(" "POINT" "(" <literal> "," <literal> ")"
                      "," "K" <integer_literal> ")"

<delete_statement> ::= "DELETE" "FROM" <identifier>
                      "WHERE" <identifier> "=" <literal>

<data_type> ::= "INT"
              | "INTEGER"
              | "SMALLINT"
              | "BIGINT"
              | "REAL"
              | "DOUBLE" "PRECISION"
              | "BOOLEAN"
              | "CHAR" "(" <integer_literal> ")"
              | "DATE"
              | "TIME"

<index_type> ::= "RTREE"
               | "BPLUS" "TREE"
               | "EXTENDIBLE" "HASHING"
               | "SEQUENTIAL" "FILE"

<primary_index_type> ::= "BPLUS" "TREE"
                       | "EXTENDIBLE" "HASHING"
                       | "SEQUENTIAL" "FILE"

<comparison_operator> ::= "<"
                       | ">"
                       | "<="
                       | ">="

<literal> ::= <integer_literal>
            | <float_literal>
            | <string_literal>
            | "TRUE"
            | "FALSE"
            | "DATE" <string_literal>
            | "TIME" <string_literal>

<identifier> ::= letra { letra | digito | "_" }

<integer_literal> ::= ["-"] digito { digito }

```

```
<float_literal> ::= ["-"] digito { digito } "." digito { digito }
```

```
<string_literal> ::= '"' { caracter } '"'
```

Esta gramática evidencia que el parser no implementa SQL completo, sino un subconjunto controlado y adaptado a las necesidades del proyecto. En particular, se priorizan instrucciones relacionadas con creación de tablas, inserción de registros, consultas con filtros, eliminación de datos, creación de índices y búsquedas espaciales mediante `POINT`, `RADIUS` y `K`. Asimismo, la gramática restringe las consultas `SELECT` al formato `SELECT * FROM tabla`, lo cual simplifica la construcción del AST y permite conectar directamente cada consulta con los operadores de búsqueda implementados por el motor.

5.2.4. Validación Sintáctica

La validación de la gramática se realiza principalmente mediante el método `expect()`, encargado de verificar que el token actual coincida exactamente con el token esperado según la producción gramatical correspondiente. En caso de inconsistencias, el sistema lanza una excepción personalizada `SyntacticError`, indicando el token esperado, el token recibido y la línea exacta donde ocurrió el error.

Además, el parser incorpora métodos auxiliares especializados para el reconocimiento de componentes complejos de la gramática, entre ellos:

- `read_literal()` para el procesamiento de literales y valores tipados.
- `read_column_type()` para el reconocimiento de tipos de datos SQL.
- `read_index_type()` para validar técnicas de indexación.
- `read_primary_index_type()` para restricciones de índices primarios.

Estos métodos permiten desacoplar reglas sintácticas específicas y mantener una arquitectura modular, facilitando futuras extensiones del lenguaje SQL soportado por el sistema.

5.3. Análisis Semántico

El análisis semántico constituye la última etapa del procesamiento de consultas antes de la ejecución sobre el motor de base de datos. Su función principal consiste en verificar que el árbol sintáctico abstracto (AST) generado durante el análisis sintáctico sea coherente con las reglas lógicas, estructurales y operacionales definidas por el sistema. A diferencia del análisis sintáctico, que únicamente valida la forma gramatical de la consulta, el análisis semántico comprueba la validez contextual de las operaciones SQL respecto al estado actual de la base de datos.

La implementación fue desarrollada mediante la clase `SemanticAnalyzer`, la cual recibe como entrada un nodo AST y ejecuta un conjunto de validaciones especializadas dependiendo del tipo de operación representada. Para ello, el sistema utiliza una estrategia basada en despacho dinámico mediante verificaciones `isinstance()`, redirigiendo automáticamente el análisis hacia funciones específicas de validación para cada tipo de nodo.

5.3.1. Validación de Operaciones SQL

El analizador semántico implementa verificaciones especializadas para todas las operaciones soportadas por el sistema:

- **CREATE TABLE:**

- Verificación de unicidad del nombre de tabla.
 - Detección de columnas duplicadas.
 - Validación de existencia de exactamente una clave primaria.
 - Verificación de restricciones `PRIMARY KEY USING`.
 - Validación de tamaños válidos para `CHAR(n)`.
 - Verificación de rutas válidas en `FROM FILE`.
- **CREATE INDEX:**
 - Verificación de existencia de la tabla.
 - Validación de unicidad del nombre del índice.
 - Verificación de columnas existentes.
 - Restricción del número de columnas según el tipo de índice.
 - Validación de índices espaciales `RTREE`.
- **INSERT:**
 - Verificación de existencia de tabla.
 - Validación de cantidad de atributos.
 - Compatibilidad entre literales y tipos de columnas.
- **SELECT y DELETE:**
 - Verificación de existencia de tablas y columnas.
 - Validación de operadores y rangos.
 - Compatibilidad de tipos en comparaciones.
 - Validación de búsquedas espaciales y `KNN`.
- **DROP TABLE y DROP INDEX:**
 - Verificación de existencia de tablas e índices antes de su eliminación.

Estas validaciones permiten evitar inconsistencias estructurales y errores de ejecución antes de que las consultas lleguen al motor de almacenamiento.

5.3.2. Verificación de Tipos de Datos

Uno de los componentes más importantes del análisis semántico es la validación de compatibilidad entre literales y tipos de datos declarados en el esquema de las tablas. Para ello, el sistema implementa la función `literal_matches_field()`, encargada de comprobar que cada valor utilizado en operaciones `INSERT`, `SELECT` o `DELETE` sea compatible con el tipo de dato de su columna correspondiente.

El sistema soporta validaciones semánticas para los siguientes tipos de datos:

- `INT`, `INTEGER`, `SMALLINT`, `BIGINT`
- `REAL`, `DOUBLE PRECISION`
- `BOOLEAN`
- `CHAR(n)`
- `DATE`

- **TIME**

En el caso de literales temporales, el analizador incorpora verificaciones semánticas adicionales sobre el formato y rango de valores. Para el tipo **DATE**, se valida el patrón **yyyy-mm-dd**, incluyendo control de meses válidos, años bisiestos y cantidad correcta de días por mes. Asimismo, para el tipo **TIME**, se verifica el formato **hh:mm:ss** y los rangos válidos para horas, minutos y segundos.

5.3.3. Validación de Consultas Espaciales

El proyecto incorpora soporte semántico para consultas espaciales basadas en índices **RTree**. Para este propósito, el analizador verifica que la tabla consultada posea efectivamente un índice espacial antes de permitir operaciones geoespaciales.

Las consultas espaciales soportadas incluyen:

- **Búsqueda por radio:**

```
SELECT * FROM tabla
WHERE ubicacion IN (POINT(x,y), RADIUS r);
```

- **Búsqueda KNN:**

```
SELECT * FROM tabla
WHERE ubicacion IN (POINT(x,y), K k);
```

Durante la validación semántica, el sistema verifica:

- Que las coordenadas del punto sean numéricas.
- Que el radio sea un valor positivo.
- Que el valor de K sea un entero positivo.
- Que la tabla posea un índice espacial **RTree**.

Estas validaciones permiten garantizar la correcta utilización de las estructuras espaciales implementadas por el motor de índices.

5.3.4. Funciones Auxiliares de Verificación

El analizador semántico incorpora además un conjunto de funciones auxiliares reutilizables orientadas a simplificar la validación contextual de consultas. Entre las más importantes destacan:

- `table_exists()` para verificar existencia de tablas.
- `column_exists()` para verificar columnas válidas.
- `validate_scalar_lookup()` para búsquedas escalares.
- `validate_spatial_lookup()` para búsquedas espaciales.
- `get_declared_column_type()` para recuperar tipos de columnas.

Estas funciones desacoplan la lógica de validación y permiten mantener una arquitectura modular, extensible y más sencilla de mantener.

5.3.5. Manejo de Errores Semánticos

Ante cualquier inconsistencia semántica detectada durante la validación, el sistema detiene inmediatamente el procesamiento de la consulta y registra un mensaje descriptivo en la variable `error_message`. Posteriormente, el parser principal encapsula este comportamiento mediante la excepción personalizada `SemanticError`, proporcionando mensajes de error claros y específicos para el usuario final.

Este mecanismo permite detectar tempranamente errores relacionados con estructuras inexistentes, incompatibilidades de tipos, índices inválidos o restricciones violadas, evitando fallos posteriores durante la ejecución de consultas sobre el motor de almacenamiento.

6. Gestión de Concurrencia y Transacciones

6.1. Protocolos de Bloqueo

El `LockManager` garantiza la consistencia de los datos mediante un protocolo de bloqueo de dos fases simplificado. La lógica de concesión de permisos se rige por la siguiente matriz de compatibilidad, la cual permite la concurrencia de múltiples lectores pero restringe el acceso durante operaciones de escritura.

Cuadro 7: Matriz de Compatibilidad de Bloqueos (S/X)

	Shared (S)	Exclusive (X)
Shared (S)	✓	X
Exclusive (X)	X	X

La Matriz de Compatibilidad resume la lógica de decisión del `LockManager` para la concesión de permisos sobre los recursos. El sistema permite que múltiples transacciones mantengan bloqueos de tipo **Shared (S)** sobre una misma página, lo que optimiza el rendimiento en operaciones de solo lectura al permitir el acceso concurrente. Sin embargo, el acceso se vuelve restrictivo cuando interviene un bloqueo **Exclusive (X)**; este modo es incompatible con cualquier otro bloqueo previo, garantizando que las operaciones de modificación se realicen de forma aislada para prevenir condiciones de carrera y asegurar la consistencia del motor.

6.2. Planificación de Transacciones

El componente `Scheduler` es el encargado de orquestar la ejecución de múltiples transacciones de forma entrelazada, siguiendo el protocolo de **Bloqueo de Dos Fases Estricto (Strict 2PL)**. Su función es transformar un flujo de texto (schedule) en una secuencia de pasos lógicos donde se evalúa la concesión de bloqueos, el manejo de esperas y la liberación de recursos.

A continuación, se presenta la traza de ejecución generada por el simulador para un escenario de conflicto típico, ilustrando el orden cronológico y el comportamiento del planificador ante la detección de bloqueos:

Cuadro 8: Traza Cronológica de Simulación y Gestión de Conflictos

Tiempo	Operación	TX	Resultado	Acción del Scheduler
t_1	R1(A)	T_1	GRANTED	Asignación de bloqueo Shared (S).
t_2	W2(A)	T_2	BLOCKED	Conflicto detectado; T_2 pasa a espera.
t_3	C1	T_1	COMMITTED	Liberación de todos los <i>locks</i> de T_1 .
t_4	—	T_2	GRANTED	Despertar de T_2 y otorgamiento de bloqueo X.

Este registro de operaciones (log) permite visualizar cómo el motor garantiza la propiedad de **aislamiento**. En el paso t_2 , el planificador identifica correctamente que la solicitud de escritura de T_2 es incompatible con el bloqueo de lectura de T_1 , forzando a la segunda transacción a esperar hasta que la primera finalice mediante un **COMMIT** en t_3 .

6.3. Ciclo de Vida de la Transacción

Este componente define la estructura y el comportamiento de las unidades de trabajo del sistema. Cada objeto **Transaction** actúa como un contenedor de metadatos que rastrea el identificador único (**tx_id**), el conjunto de bloqueos adquiridos (**locks_held**) y un registro cronológico (*log*) de sus operaciones individuales.

El ciclo de vida se gestiona mediante una máquina de estados finitos que transita entre las siguientes etapas:

- **ACTIVE:** Estado inicial de procesamiento.
- **WAITING:** Estado de suspensión cuando existe un conflicto de recursos.
- **COMMITTED / ABORTED:** Estados terminales que representan la finalización exitosa o la cancelación de la transacción, desencadenando la liberación inmediata de todos los bloqueos asociados.

cion7y9 · TEX Copy

7. Resultados Experimentales y Discusión

7.1. Pruebas de rendimiento con volúmenes de datos

Las pruebas de rendimiento se ejecutaron sobre el motor implementado, midiendo los accesos a disco (lecturas y escrituras de páginas de 4 KB) y el tiempo de respuesta en milisegundos para operaciones representativas de cada estructura de indexación. La Tabla 9 resume los resultados obtenidos experimentalmente.

Cuadro 9: Resultados experimentales de rendimiento por estructura de indexación

Índice	Consulta	Rows	Disk Reads	Total I/O	Tiempo (ms)
B+ Tree	WHERE id = 1 (búsqueda puntual)	1	3	3	7.94
B+ Tree	WHERE salario BETWEEN 4000 AND 7000	0	3	3	0.56
Ext. Hashing	WHERE codigo = 202 (búsqueda exacta)	1	2	2	12.27
Sequential File	WHERE monto BETWEEN 100 AND 1000	0	1	1	0.26
R-Tree (Radio)	RADIUS 500 km desde Lima	11	56	56	4.76
R-Tree (KNN)	KNN=5 desde Madrid	5	26	26	2.74

Los resultados validan el comportamiento teórico de cada estructura. El Extendible Hashing accede a solo **2 páginas** para localizar un registro exacto (directorío + bucket), confirmando su complejidad $O(1)$. El B+ Tree requiere **3 páginas** para descender por el árbol hasta la hoja correspondiente, consistente con $O(\log_4 N)$ para un árbol de poca profundidad. El Sequential

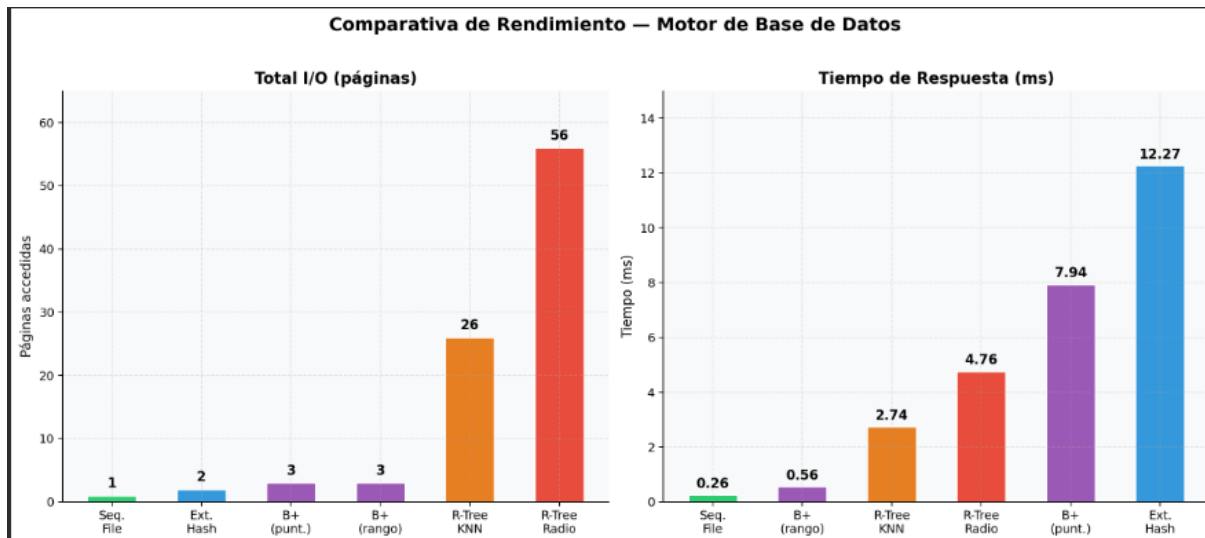


Figura 2: Comparativa de Rendimiento - Motor de Base de datos

File accede a **1 página** en la búsqueda por rango gracias a la búsqueda binaria sobre el archivo principal ordenado. El R-Tree, al operar sobre 60 puntos geográficos distribuidos globalmente, requiere más accesos de página debido a la exploración de nodos internos con MBRs que se solapan en distribuciones dispersas.

7.2. Gráficos comparativos y discusión técnica

La Figura 2 presenta la comparativa de accesos a disco (Total I/O) y la Figura 1 muestra el tiempo de respuesta para cada operación evaluada.

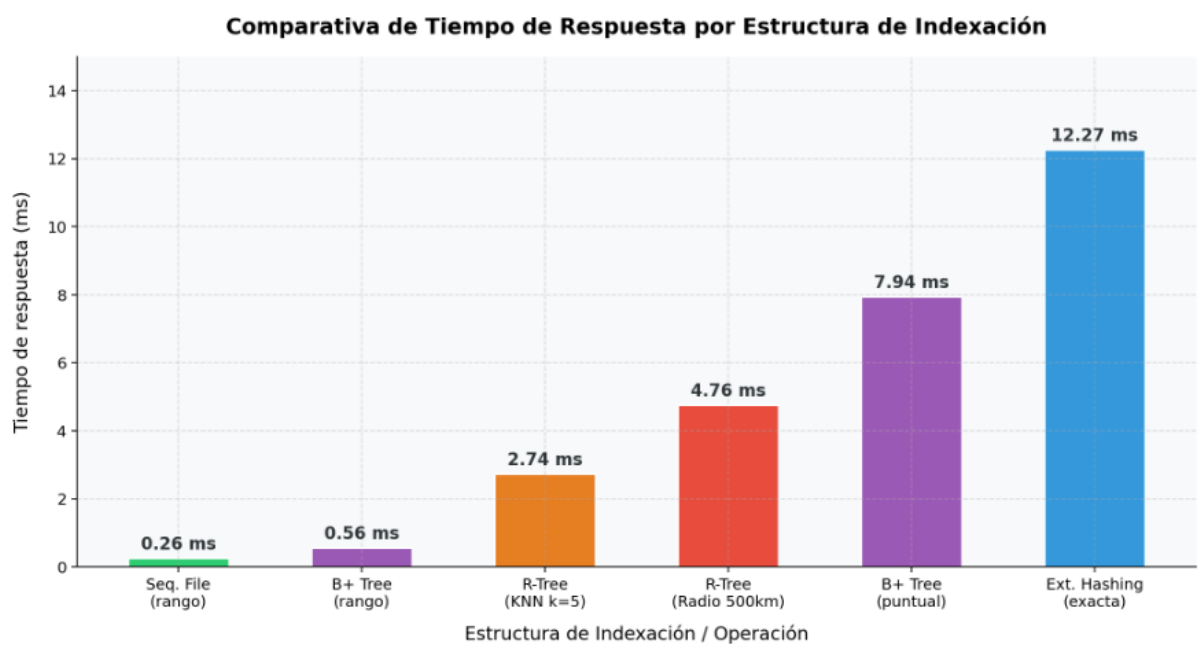


Figura 1: Estructura de Indexación/ Operación

Discusión de resultados Los resultados experimentales son coherentes con el análisis teórico presentado en la Sección 4. Se destacan los siguientes hallazgos:

- **Sequential File** registra el menor número de accesos a disco (1 página) y el menor tiempo de respuesta (0.26 ms) para la búsqueda por rango, dado que la tabla **ventas** es pequeña y el archivo principal cabe en una sola página, permitiendo que la búsqueda binaria resuelva la consulta en un único acceso.
- **Extendible Hashing** confirma su complejidad $O(1)$ con solo 2 accesos (directorio + bucket) para localizar el registro exacto. Sin embargo, el tiempo de respuesta fue de 12.27 ms, el más alto del experimento, lo cual se atribuye a la sobrecarga de inicialización de la función hash y la deserialización del bucket en Python.
- **B+ Tree** requiere 3 accesos en ambas consultas, validando la profundidad $h = \lceil \log_4 N \rceil$ para $N = 5$ registros. La búsqueda por rango resulta más rápida (0.56 ms) que la puntual (7.94 ms) debido a diferencias en el tiempo de serialización de la respuesta.
- **R-Tree (RADIUS)** accede a 56 páginas para 60 puntos distribuidos globalmente con un radio de 500 km. El alto número de accesos se debe a que los MBRs de los nodos internos intersectan el área de búsqueda al estar los datos dispersos geográficamente, forzando la exploración de múltiples ramas del árbol.
- **R-Tree (KNN)** accede a 26 páginas para retornar los 5 vecinos más cercanos a Madrid. La cola de prioridad del algoritmo KNN permite podar ramas lejanas de forma más eficiente que el RADIUS, resultando en menos accesos y menor tiempo (2.74 ms).

Discusión de resultados. Los resultados experimentales son coherentes con el análisis teórico presentado en la Sección 4. Se destacan los siguientes hallazgos:

- **Sequential File** registra el menor número de accesos a disco (1 página) y el menor tiempo de respuesta (0.26 ms) para la búsqueda por rango, lo cual se explica porque la tabla **ventas** es pequeña y el archivo principal cabe en una sola página, permitiendo que la búsqueda binaria resuelva la consulta en un único acceso.
- **Extendible Hashing** confirma su complejidad $O(1)$ con solo 2 accesos (directorio + bucket) para localizar el registro exacto. Sin embargo, el tiempo de respuesta fue de 12.27 ms, ligeramente mayor que el B+ Tree, lo cual se atribuye a la sobrecarga de la función hash y la deserialización del bucket en Python.
- **B+ Tree** requiere 3 accesos para descender por el árbol hasta la hoja en ambas consultas (búsqueda puntual y por rango), validando la profundidad $h = \lceil \log_4 N \rceil$ para $N = 5$ registros.
- **R-Tree (RADIUS)** accede a 56 páginas para procesar 60 puntos distribuidos globalmente con un radio de 500 km. El alto número de accesos se debe a que los MBRs de los nodos internos intersectan el área de búsqueda, forzando la exploración de múltiples ramas. Este comportamiento es esperado cuando los datos están distribuidos globalmente y el radio de búsqueda es pequeño en comparación con la extensión total del dataset.
- **R-Tree (KNN)** accede a 26 páginas para retornar los 5 vecinos más cercanos a Madrid. La cola de prioridad del algoritmo KNN permite podar ramas lejanas de forma más eficiente que el RADIUS, resultando en menos accesos y menor tiempo (2.74 ms).

8. Interfaz Gráfica y Capturas

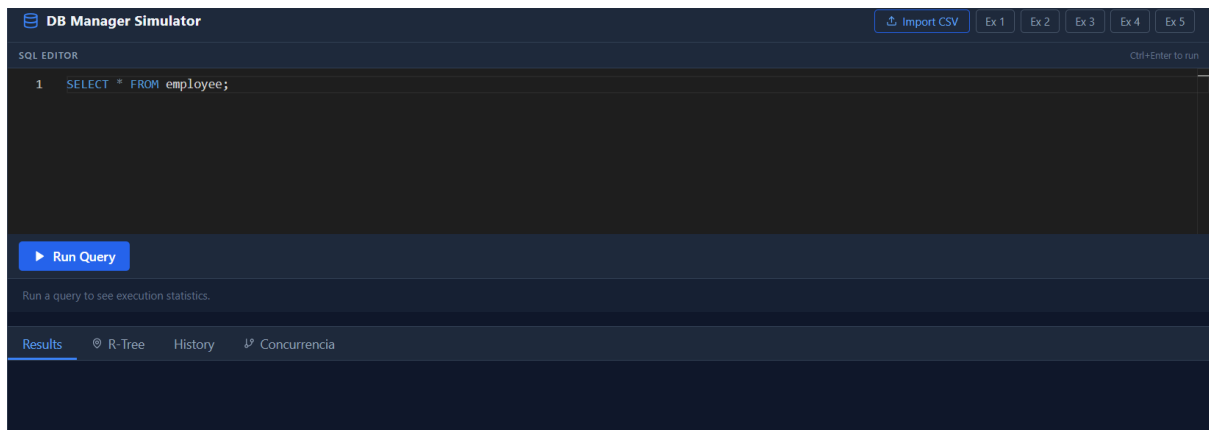


Figura 3: Vista general de la interfaz del DB Manager Simulator.

8.1. Visualización de la arquitectura y ejecución SQL

8.1.1. B+ Tree

La tabla de empleados fue creada usando un índice primario B+ Tree, el cual organiza los registros en un árbol balanceado que permite búsquedas, inserciones y eliminaciones en tiempo $O(\log n)$.

```
CREATE TABLE empleados (  
    id      INT PRIMARY KEY USING BPLUS TREE,  
    nombre CHAR(50),  
    salario INT  
);
```

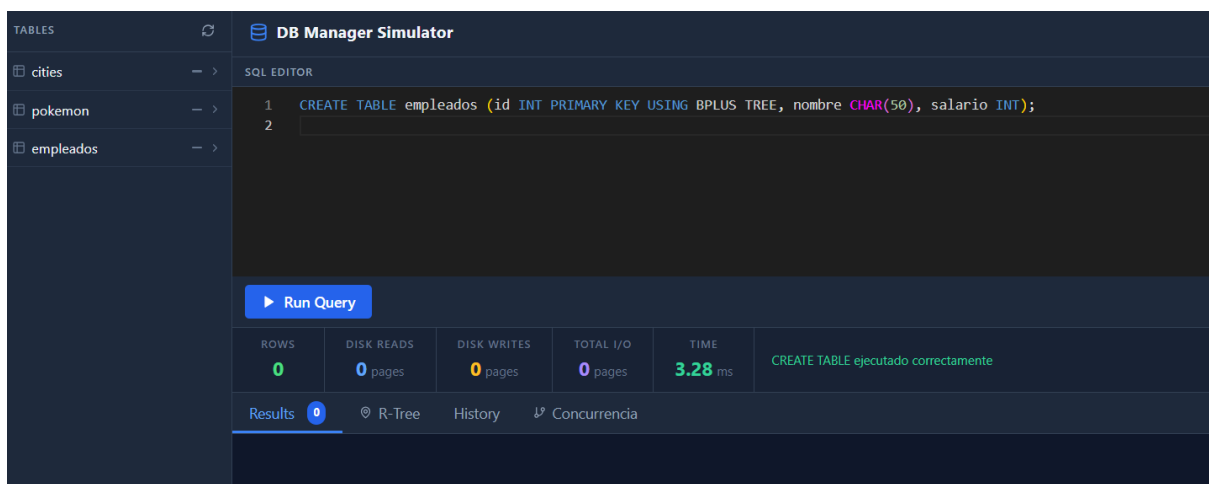


Figura 4: Creación de tabla con índice B+ Tree.

TABLES	DB Manager Simulator	SQL EDITOR
cities		1 SELECT * FROM empleados;
pokemon		
empleados		
id	INT	
nombre	CHAR	
salario	INT	
Drop		
	Run Query	
	ROWS	DISK READS
	5	4 pages
		DISK WRITES
		0 pages
		TOTAL I/O
		4 pages
		TIME
		0.39 ms
	Results	R-Tree
	History	Concurrencia
	5 rows	
	ID	NOMBRE
	1	Ana García
	2	Luis Torres
	3	Maria Lopez
	4	Carlos Ruiz
	5	Sofia Diaz
		SALARIO
		5000
		7000
		4500
		9000
		6500

Figura 5: Inserciones y consultas sobre tabla con B+ Tree.

8.1.2. Extendible Hashing

El hashing extensible utiliza un directorio dinámico que se duplica cuando los buckets se llenan, garantizando acceso en tiempo $O(1)$ promedio para búsquedas por clave exacta.

```
CREATE TABLE productos (
    codigo INT PRIMARY KEY USING EXTENDIBLE HASHING,
    nombre CHAR(50),
    precio INT
);
```

TABLES	DB Manager Simulator	SQL EDITOR
cities		1 CREATE TABLE productos (codigo INT PRIMARY KEY USING EXTENDIBLE HASHING, nombre CHAR(50), precio INT);
pokemon		
empleados		
id	INT	
nombre	CHAR	
salario	INT	
Drop		
productos		
	Run Query	
	ROWS	DISK READS
	0	0 pages
		DISK WRITES
		2 pages
		TOTAL I/O
		2 pages
		TIME
		6.58 ms
	Results	R-Tree
	History	Concurrencia
	0	

Figura 6: Creación de tabla con índice Extendible Hashing.

ROWS	DISK READS	DISK WRITES	TOTAL I/O	TIME	
4	2 pages	0 pages	2 pages	3.83 ms	
Results 4 R-Tree History Concurrency					
4 rows					
CODIGO	NOMBRE		PRECIO		
101	Laptop		2500		
202	Mouse		50		
303	Teclado		120		
404	Monitor		800		

Figura 7: Inserciones y consultas sobre tabla con Extendible Hashing.

8.1.3. Sequential File

El archivo secuencial mantiene los registros ordenados por clave primaria en disco. Utiliza un archivo auxiliar de desbordamiento que se reconstruye físicamente cuando alcanza un umbral K , optimizando las búsquedas por rango.

```
CREATE TABLE ventas (
    id          INT PRIMARY KEY USING SEQUENTIAL FILE,
    producto    CHAR(50),
    monto       INT
);
```

TABLES		DB Manager Simulator				
cities	— >	SQL EDITOR				
pokemon	— >	1 CREATE TABLE ventas (id INT PRIMARY KEY USING SEQUENTIAL FILE, producto CHAR(50), monto INT);				
empleados	— >					
productos	— v					
codigo	INT					
nombre	CHAR	Run Query				
precio	INT					
Drop						
ventas	— >	ROWS	DISK READS	DISK WRITES	TOTAL I/O	TIME
		0	0 pages	0 pages	0 pages	5.81 ms
		CREATE TABLE ejecutado correctamente				
		Results 0 R-Tree History Concurrency				

Figura 8: Creación de tabla e inserciones con Sequential File.

8.1.4. R-Tree

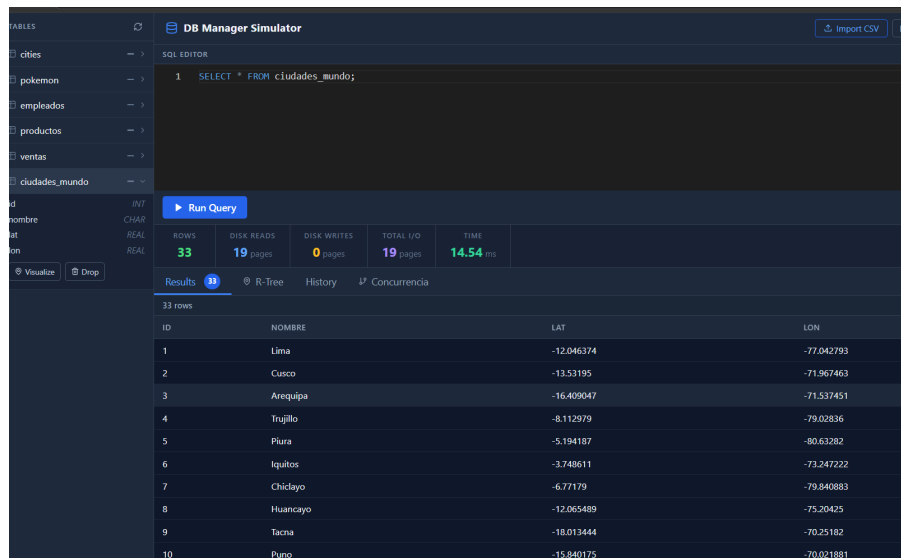
El R-Tree es un índice espacial que agrupa objetos geográficos mediante rectángulos mínimos envolventes (MBR). Soporta consultas por radio (RADIUS) y k-vecinos más cercanos (KNN).

```
CREATE TABLE ciudades_mundo (
    id          INT PRIMARY KEY USING BPLUS TREE,
```

```

    nombre CHAR(50),
    lat     REAL,
    lon     REAL
);
CREATE INDEX idx_rtree ON ciudades_mundo (lat, lon) USING RTREE;

```



ID	NOMBRE	LAT	LON
1	Lima	-12.046374	-77.042793
2	Cusco	-13.53195	-71.967463
3	Arequipa	-16.409047	-71.537451
4	Trujillo	-8.112979	-79.02836
5	Piura	-5.194187	-80.63282
6	Iquitos	-3.748611	-73.247222
7	Chiclayo	-6.77179	-79.840883
8	Huancayo	-12.065489	-75.20425
9	Tacna	-18.013444	-70.25182
10	Puno	-15.840175	-70.021881

Figura 9: Inserciones de ciudades del mundo en la tabla con índice R-Tree.

Consulta por radio desde Lima (300 km):

```

SELECT * FROM ciudades_mundo
WHERE lat IN (POINT(-12.0, -77.0), RADIUS 300.0);

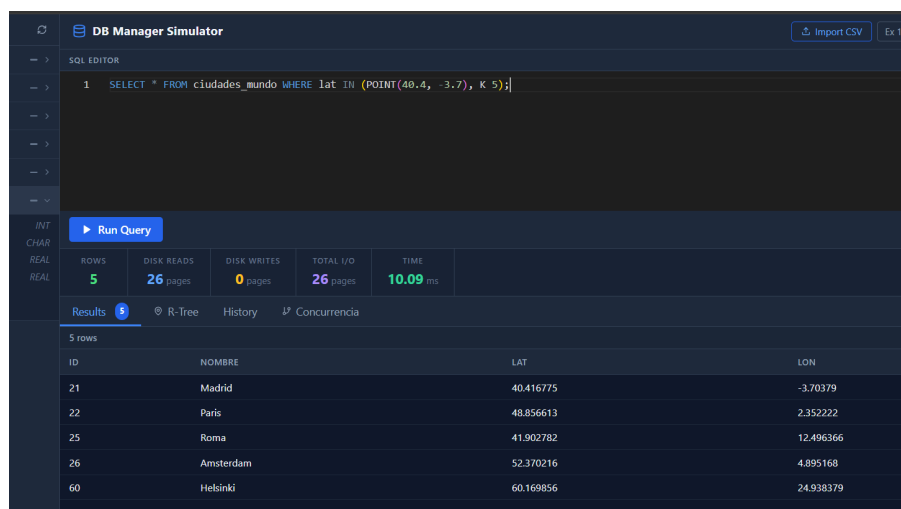
```

KNN — 5 ciudades más cercanas a Madrid:

```

SELECT * FROM ciudades_mundo
WHERE lat IN (POINT(40.4, -3.7), KNN 5);

```



ID	NOMBRE	LAT	LON
21	Madrid	40.416775	-3.70379
22	Paris	48.856613	2.352222
25	Roma	41.902782	12.496366
26	Amsterdam	52.370216	4.895168
60	Helsinki	60.169856	24.938379

Figura 11: Consulta KNN: 5 ciudades más cercanas a Madrid, visualizadas con líneas hacia el punto de consulta.

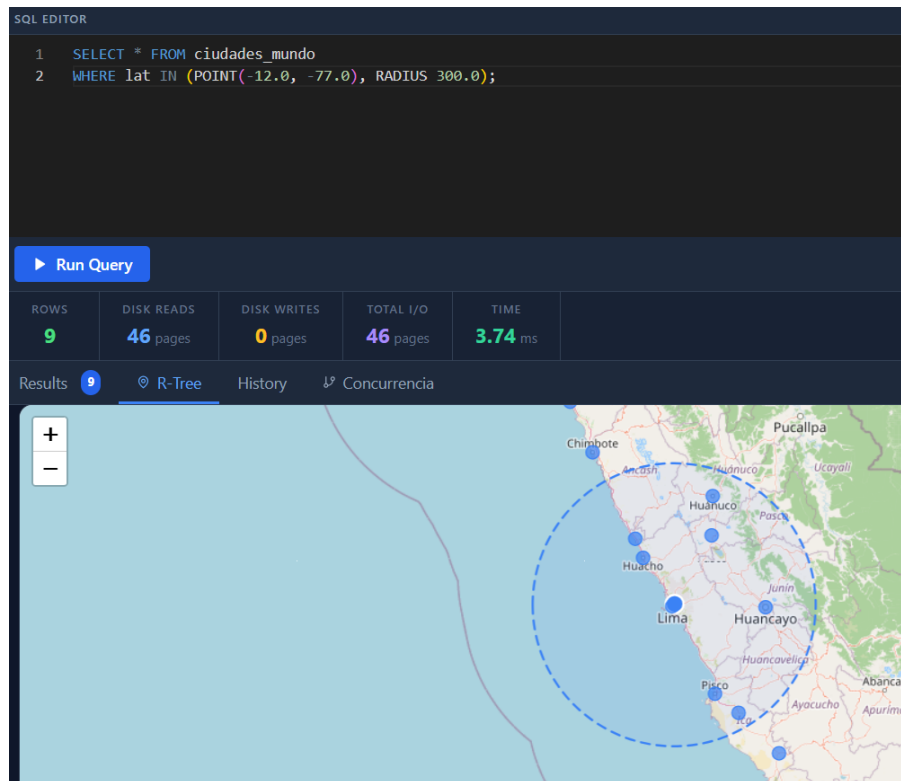


Figura 10: Área por radio de 300km

KNN — 3 ciudades más cercanas a Tokio:

`SELECT * FROM ciudades_mundo WHERE lat IN (POINT(40.4, -3.7), KNN 5);`

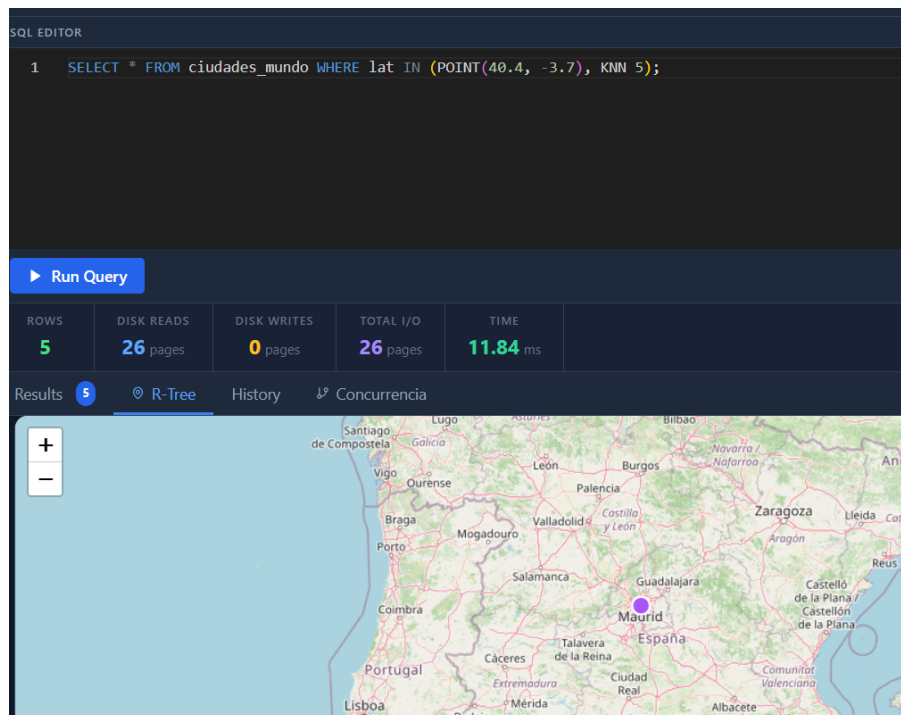


Figura 12: KNN

Importación de datos vía CSV: El sistema permite cargar datos masivamente desde archivos CSV mediante la interfaz gráfica, sin necesidad de insertar registros uno a uno.

Importar desde CSV
sudamerica.csv — 4 columnas detectadas

Nombre de tabla: Tipo de índice: **B+ Tree** (dropdown)
Búsqueda exacta y por rango

Columnas detectadas

Columna	Tipo	Tamaño	PK
id	INT	4B	☒
nombre	CHAR	21	☐
lat	REAL	8B	☐
lon	REAL	8B	☐

Índices secundarios (opcional) [+ Agregar índice](#)
Sin índices secundarios

Vista previa (primeras 5 filas)

id	nombre	lat	lon
1	Lima	-12.046374	-77.042793
2	Cusco	-13.531950	-71.967463
3	Arequipa	-16.409047	-71.537451
4	Trujillo	-8.112070	-79.028360

[← Cambiar archivo](#) [⬆ Crear tabla e importar](#)

Figura 13: Modal de importación de CSV en la interfaz gráfica.

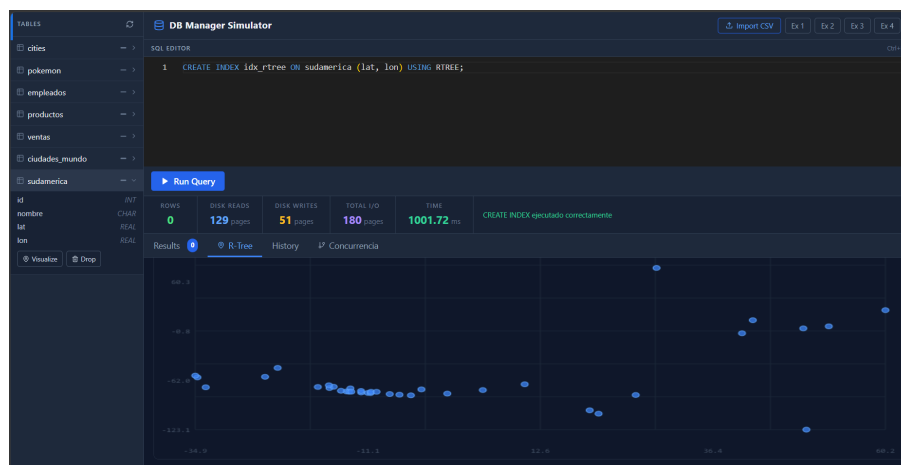


Figura 14: Datos cargados correctamente desde el archivo CSV.

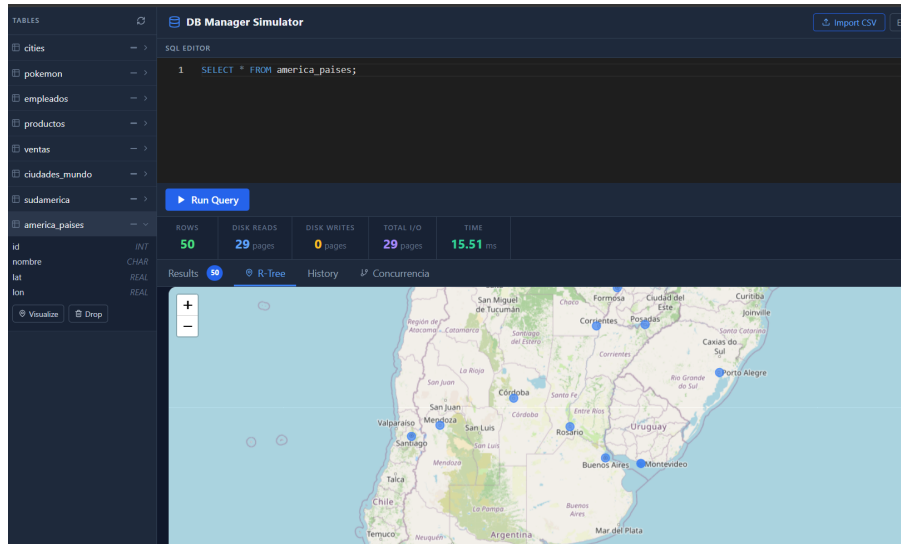


Figura 15: Mapa R-tree

8.1.5. Manejo de errores

El sistema detecta y muestra errores al usuario en tiempo real, incluyendo claves duplicadas, tipos de dato incorrectos y sintaxis SQL inválida.

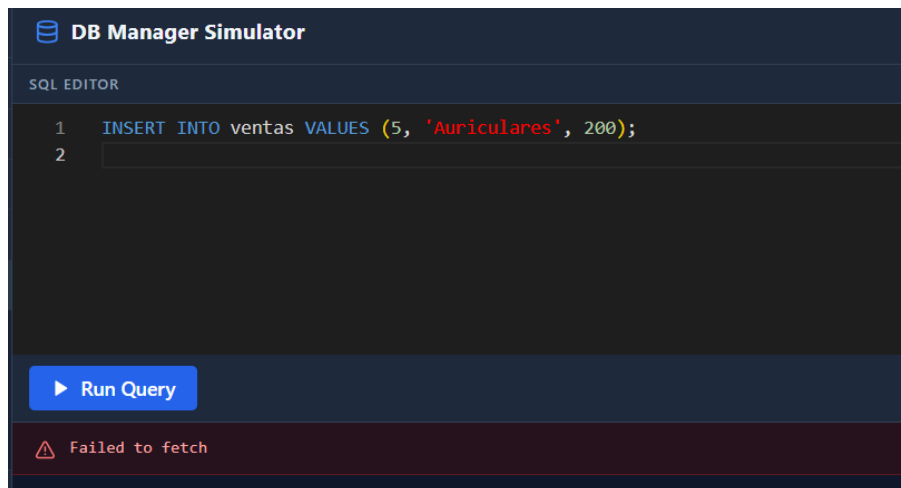


Figura 16: Error mostrado al intentar insertar una clave primaria duplicada.

9. Conclusiones

El desarrollo de este motor de base de datos relacional desde cero permitió comprender en profundidad los fundamentos del almacenamiento paginado, la indexación avanzada y el control de concurrencia. A continuación se presentan las conclusiones más relevantes del proyecto:

1. **El costo de I/O es la métrica determinante del rendimiento.** La implementación sobre páginas de 4 KB confirmó que el número de accesos a disco es el cuello de botella principal. Estructuras como el Extendible Hashing minimizan este costo a $O(1)$ para búsquedas exactas, mientras que el B+ Tree ofrece un balance óptimo entre búsquedas puntuales ($O(\log_4 N)$) y por rango ($O(\log_4 N + L)$).

2. **No existe una estructura de indexación universalmente óptima.** Cada técnica implementada presenta ventajas y limitaciones claras según el tipo de consulta. El Extendible Hashing es insuperable para igualdad exacta pero no soporta búsquedas por rango. El Sequential File es eficiente para cargas masivas y rangos sobre datos ordenados, pero se degrada con inserciones frecuentes. El R-Tree es indispensable para datos espaciales, pero su rendimiento depende fuertemente de la distribución geográfica de los puntos y la calidad de los MBRs.
3. **El R-Tree demostró ser una estructura poderosa pero sensible a la distribución.** Las consultas espaciales por radio y KNN funcionaron correctamente sobre 60 ciudades del mundo, retornando resultados geográficamente precisos mediante la fórmula de Haversine. Sin embargo, la distribución global de los puntos genera MBRs amplios que reducen la eficacia de la poda, incrementando el número de accesos. Para datos geográficamente concentrados (por ejemplo, solo ciudades de Perú), el rendimiento del R-Tree mejoraría significativamente.
4. **El parser SQL es el componente que define la usabilidad del sistema.** La implementación de un analizador léxico-sintáctico propio permitió traducir sentencias SQL en ASTs ejecutables directamente sobre las estructuras de indexación. La gestión de errores en tiempo real (claves duplicadas, tipos incorrectos, sintaxis inválida) demostró ser crítica para la experiencia del usuario. Sin embargo, la gramática soportada es un subconjunto reducido de SQL estándar, lo cual es una limitación inherente del alcance del proyecto.
5. **El control de concurrencia garantiza la consistencia bajo acceso simultáneo.** La implementación del protocolo Strict 2PL mediante el LockManager permitió gestionar correctamente los conflictos entre transacciones concurrentes. La matriz de compatibilidad S/X asegura que múltiples lectores puedan operar simultáneamente, mientras que las escrituras se realizan de forma aislada, previniendo condiciones de carrera sobre las páginas compartidas.
6. **La interfaz gráfica potencia la comprensión del motor.** El desarrollo de un frontend con React y Vite, integrado con un visualizador de R-Tree basado en mapas reales (Leaflet/OpenStreetMap), permitió observar el comportamiento espacial del índice de forma intuitiva. La visualización del radio de búsqueda, los puntos resultado y las líneas KNN sobre el mapa geográfico real constituyó una herramienta de validación visual efectiva para las consultas espaciales.
7. **Docker garantiza la reproducibilidad del entorno.** La containerización del sistema mediante Docker permitió encapsular el backend (FastAPI/Python), el frontend (React/Vite) y el servidor web (Nginx) en una imagen única y reproducible. El uso de volúmenes persistentes asegura que los datos sobrevivan al ciclo de vida del contenedor, cumpliendo el requisito de persistencia del motor.

Trabajo futuro. Como extensiones naturales del sistema se identifican: la implementación de joins entre tablas, el soporte de transacciones ACID completas con write-ahead logging (WAL), la optimización del parser para soportar subconsultas y agregaciones (GROUP BY, COUNT, AVG), y la incorporación de un buffer pool manager para reducir los accesos físicos a disco mediante caché en memoria.